

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Mohan Kumar J	Reg. No.:	192424060
Section / Batch:	2024 [AI&DS]	Date:	23-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Minimum Spanning Tree Using Prim's Algorithm

1. Introduction

A Minimum Spanning Tree (MST) is a tree that connects all the vertices of a weighted, connected graph with the minimum possible total cost.

Prim's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree. It starts from a given vertex and repeatedly selects the minimum-weight edge that connects a vertex already included in the tree to a new vertex.

The main objectives are:

- Connect all nodes.
- Minimize the total edge cost.
- Avoid cycles.
- Build the network efficiently.

Applications include computer network design, road and transportation networks, electrical power distribution, telephone networks, water pipeline networks, and communication systems.

2. Problem Understanding

Consider a weighted graph:

$$G = (V, E)$$

where V represents the set of vertices or nodes and E represents the set of edges. Each edge has a weight representing its cost or distance.

The objective is to connect all vertices with the minimum total edge weight.

For a graph containing V vertices, the MST always contains:

$$V - 1 \text{ edges}$$

A valid MST must:

- Connect all vertices.
- Contain exactly $V - 1$ edges.
- Contain no cycles.
- Have the minimum possible total weight.

The graph should normally be weighted, undirected, and connected.

3. Prim's Algorithm

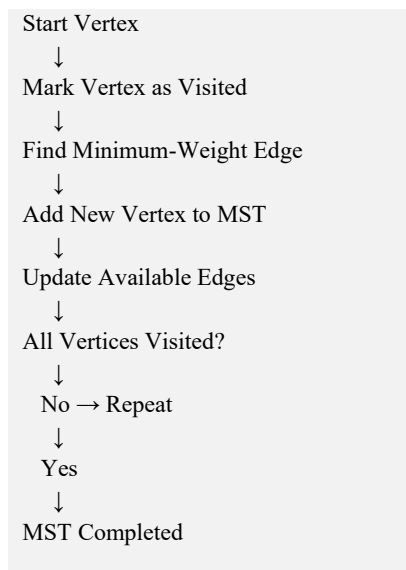
Prim's Algorithm follows a greedy approach. It starts from a specified vertex and grows the spanning tree one edge at a time.

Minimum Spanning Tree – Prim's Algorithm

Steps of Prim's Algorithm:

1. Select the starting vertex.
2. Mark the vertex as visited.
3. Find all edges connecting visited vertices to unvisited vertices.
4. Select the edge with the minimum weight.
5. Add the selected edge and new vertex to the MST.
6. Repeat until all vertices are included.
7. Calculate the total weight of the selected edges.

The main greedy rule is: Always select the minimum-weight edge that connects the current MST to an unvisited vertex.



4. Step-by-Step Example

Consider the following weighted graph:

Edge	Weight
A-B	2
A-C	3
B-C	1
B-D	4
C-D	5
C-E	6
D-E	2

Let the starting vertex be A.

Step 1: Start with A. The available edges are A-B = 2 and A-C = 3. Select A-B = 2.

Step 2: Visited vertices are {A, B}. The available edges are A-C = 3, B-C = 1, and B-D = 4. Select B-C = 1.

Step 3: Visited vertices are {A, B, C}. The available edges are B-D = 4, C-D = 5, and C-E = 6. Select B-D = 4.

Step 4: Visited vertices are {A, B, C, D}. The available edges are C-E = 6 and D-E = 2. Select D-E = 2.

Now all five vertices are connected.

Selected Edge	Weight
A-B	2
B-C	1
B-D	4
D-E	2

Total cost = $2 + 1 + 4 + 2 = 9$.

Therefore, the MST cost is 9. Since there are 5 vertices, the MST contains $5 - 1 = 4$ edges.

5. Python Implementation

```
def prim_mst(graph, start):  
  
    visited = set()  
    mst = []  
    total_cost = 0  
  
    visited.add(start)  
  
    while len(visited) < len(graph):  
  
        minimum_edge = None  
        minimum_weight = float('inf')  
  
        for u in visited:  
            for v, weight in graph[u]:
```

```

        if v not in visited and weight < minimum_weight:
            minimum_weight = weight
            minimum_edge = (u, v, weight)

    if minimum_edge is None:
        print("Graph is disconnected.")
        return

    u, v, weight = minimum_edge

    mst.append((u, v, weight))
    total_cost += weight
    visited.add(v)

print("Minimum Spanning Tree:")

for u, v, weight in mst:
    print(f"{u} - {v} : {weight}")

print("Total MST Cost:", total_cost)

graph = {
    'A': [('B', 2), ('C', 3)],
    'B': [('A', 2), ('C', 1), ('D', 4)],
    'C': [('A', 3), ('B', 1), ('D', 5), ('E', 6)],
    'D': [('B', 4), ('C', 5), ('E', 2)],
    'E': [('C', 6), ('D', 2)]
}

prim_mst(graph, 'A')

```

Expected Output:

Minimum Spanning Tree:

A - B : 2

B - C : 1

B - D : 4

D - E : 2

Total MST Cost: 9

6. Algorithm Analysis and Complexity

The efficiency of Prim's Algorithm depends on the graph representation.

Using an Adjacency Matrix

When an adjacency matrix is used, the algorithm checks the vertices to find the minimum edge.

Time Complexity: $O(V^2)$

Space Complexity: $O(V^2)$

This approach is suitable for smaller or dense graphs.

Using an Adjacency List and Priority Queue

For a more efficient implementation, an adjacency list with a priority queue can be used.

Time Complexity: $O(E \log V)$

Space Complexity: $O(V + E)$

Here, V is the number of vertices and E is the number of edges. This approach is more suitable for large and sparse graphs.

7. Competitive Performance

Prim's Algorithm provides good performance because it uses a greedy strategy and does not examine every possible spanning tree.

For a graph with V vertices, the resulting MST always contains $V - 1$ edges.

Advantages:

- Efficient for large graphs.
- Simple to understand.
- Easy to implement.
- Produces a minimum spanning tree for a connected weighted graph.
- Works well with priority queues.
- Useful for network optimization.

Comparison with Kruskal's Algorithm:

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Grows one tree	Selects edges globally
Main structure	Priority Queue	Disjoint Set
Suitable for	Dense graphs	Sparse graphs
Selection	Minimum edge from current tree	Globally minimum edge
Cycle handling	Visited vertices	Union-Find

8. Test Case Handling

The algorithm should be tested using different types of inputs.

Test Case	Input	Expected Result
TC01	Normal connected graph	Valid MST generated
TC02	Different starting vertex	Valid MST generated
TC03	Equal edge weights	Any valid minimum MST accepted
TC04	Disconnected graph	Report graph is disconnected
TC05	Single vertex	MST cost is 0
TC06	Negative edge weights	MST generated correctly

If the graph is disconnected, all vertices cannot be connected using a single spanning tree. The algorithm should report that the graph is disconnected.

9. Validation and Results

The generated MST can be validated using the following conditions:

- All vertices must be connected.
- The number of edges must be $V - 1$.
- The MST must contain no cycles.
- The total edge weight must be minimum.

For the example, the selected edges are A-B, B-C, B-D, and D-E. The number of edges is 4, which equals $5 - 1$. The total cost is $2 + 1 + 4 + 2 = 9$.

Validation	Result
All vertices connected	Yes
Correct number of edges	Yes
Cycles present	No
Minimum cost	9
MST generated successfully	Yes

10. Conclusion

Prim's Algorithm is an efficient greedy algorithm for constructing a Minimum Spanning Tree of a connected, weighted, undirected graph.

The algorithm starts from a given vertex and repeatedly selects the minimum-weight edge that connects the current tree to an unvisited vertex. It continues until all vertices are connected.

For the given example, the selected edges are A-B, B-C, B-D, and D-E, and the minimum total cost is 9.

The algorithm has a time complexity of $O(V^2)$ when an adjacency matrix is used and $O(E \log V)$ when an adjacency list with a priority queue is used.

Prim's Algorithm is widely applicable to network design, transportation systems, electrical networks, communication systems, and other minimum-cost connection problems.

Overall, Prim's Algorithm provides a simple, efficient, and reliable solution for connecting all nodes with the minimum possible total cost.

ASSESSMENT CO4-AT1-Coding Challenge Task

[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1

Minimum Spanning Tree (Prim)
100 marks

1/1 answered

Q1 Minimum Spanning Tree (Prim)

100 marks

A system must construct a network that connects all nodes with the least total cost, starting from a given node. Using Prim's algorithm, iteratively select the minimum weight edge to expand the tree until all nodes are connected.

Your Code

```
1 import sys
2 import heapq
3 def prim_mst(n,graph,start):
4     visited=[False]*(n+1)
5     min_heap=[(0,start,-1)]
6     mst=[]
7     total_cost=0
8     while min_heap and len(mst)<n-1:
9         weight,u,parent=heapq.heappop(min_heap)
10        if visited[u]:
11            continue
12        visited[u]=True
13        if parent !=-1:
14            mst.append((parent,u,weight))
15            total_cost+=weight
16        for v,w in graph[u]:
17            if not visited[v]:
18                heapq.heappush(min_heap,(w,v,u))
19        if len(mst)!=n-1:
20            return None,0
21        return mst,total_cost
22 def main():
23     data=list(map(int,sys.stdin.read().split()))
24     if not data:
25         return
26     index=0
27     n=data[index]
28     m=data[index+1]
29     index+=2
30     graph=[[] for _ in range(n+1)]
31     for _ in range(m):
32         u=data[index]
33         v=data[index+1]
34         w=data[index+2]
35         index+=3
36         graph[u].append((v,w))
37         graph[v].append((u,w))
38     start=data[index]
39     mst,total_cost=prim_mst(n,graph,start)
40
41     if mst is None:
42         print("\n MST does not exist.")
43         print("The graph is disconnected")
44         return
45     for u,v,w in mst:
46         print(f"{u}-{v}:(w)")
47     print("Minimum Cost: ",total_cost)
48 main()
```

Submitted

Input

```
5 6
1 2 2
1 3 3
2 3 1
```

Output

```
1-2:2
2-3:1
3-4:2
4-5:3
Minimum Cost: 8
```